Q1. Greedy Best-First Search for Website Design (and Chatbot Design)

> **Question.** Use greedy best-first search (GBFS) for website design. What is the heuristic? Map node expansion and traversal to design changes. Discuss convergent and divergent approaches. Extend the idea to chatbot design with multiple trees. Can each member of a population be a decision tree (Diagram 1)? Can the heuristic be dynamic? Give a step-by-step approach, pseudocode and one worked example.

1. The idea in plain words

A website can be improved one change at a time: a new layout, simpler navigation, faster images. Each version of the site is a **state**. GBFS looks at all versions it can reach with one more change and always jumps to the one that *looks* closest to a great site. "Looks closest" is the job of the **heuristic h(n)**. GBFS does not care how many changes it already made (it ignores path cost). It only asks: *which version seems nearest to the goal right now?*

2. Assumptions

- A site design has 5 features: layout, navigation, hero section, call-to-action (CTA) and images, each with 3 options.
- The UX team rates every option for **usability, speed and mobile-friendliness** (0 to 10), for example from past A/B tests.
- The starting site is old-fashioned: classic layout, mega-menu, slider, no CTA, heavy images.
- The goal is a UX score of at least 80/100, which means **h(n) ≤ 20**.

3. Mapping search to website design

Search term	Meaning in website design
State / node	One complete version of the website (a choice for each of the 5 features)
Start state S0	The current live website
Action (operator)	One design change, e.g. `images → lazy-loaded`
Node expansion	Listing every version reachable with **one** change (10 children per node here)
Traversal	The sequence of accepted changes, i.e. the redesign roadmap
Heuristic h(n)	Estimated gap between this version and an ideal site (lower is better)
Goal test	h(n) ≤ 20 (UX score ≥ 80)

The heuristic

$UX(n) = 100 \times \Sigma_{features}\ (w_u \cdot usability + w_s \cdot speed + w_m \cdot mobile) / (10 \times 5)$ and **$h(n) = 100 - UX(n)$**.

In week 1 the weights are w = (0.50, 0.30, 0.20). The option ratings (usability, speed, mobile) are:

Feature	Option 1	Option 2	Option 3
Layout	classic (5,6,4)	grid (7,6,6)	single-page (8,7,9)
Navigation	mega-menu (6,4,2)	simple (8,8,7)	hamburger (6,8,10)
Hero	slider (4,3,4)	static-image (7,8,7)	short-video (8,4,6)
CTA	none (2,10,5)	one-button (8,10,8)	sticky (9,9,9)
Images	heavy (6,1,2)	compressed (6,8,7)	lazy-loaded (7,10,9)

4. Step-by-step approach

1. Describe the live site as the start state S0 and compute h(S0).
2. Put S0 in the OPEN list, which is sorted by h (lowest first).
3. Take the version with the lowest h out of OPEN. If h ≤ 20, stop: this is the design to ship.
4. Otherwise **expand** it: make every one-change variant and compute h for each.
5. Add the unseen variants to OPEN and remember the visited ones, so we never redo a design.
6. Repeat from step 3. Optionally, update the heuristic weights whenever new analytics arrive.

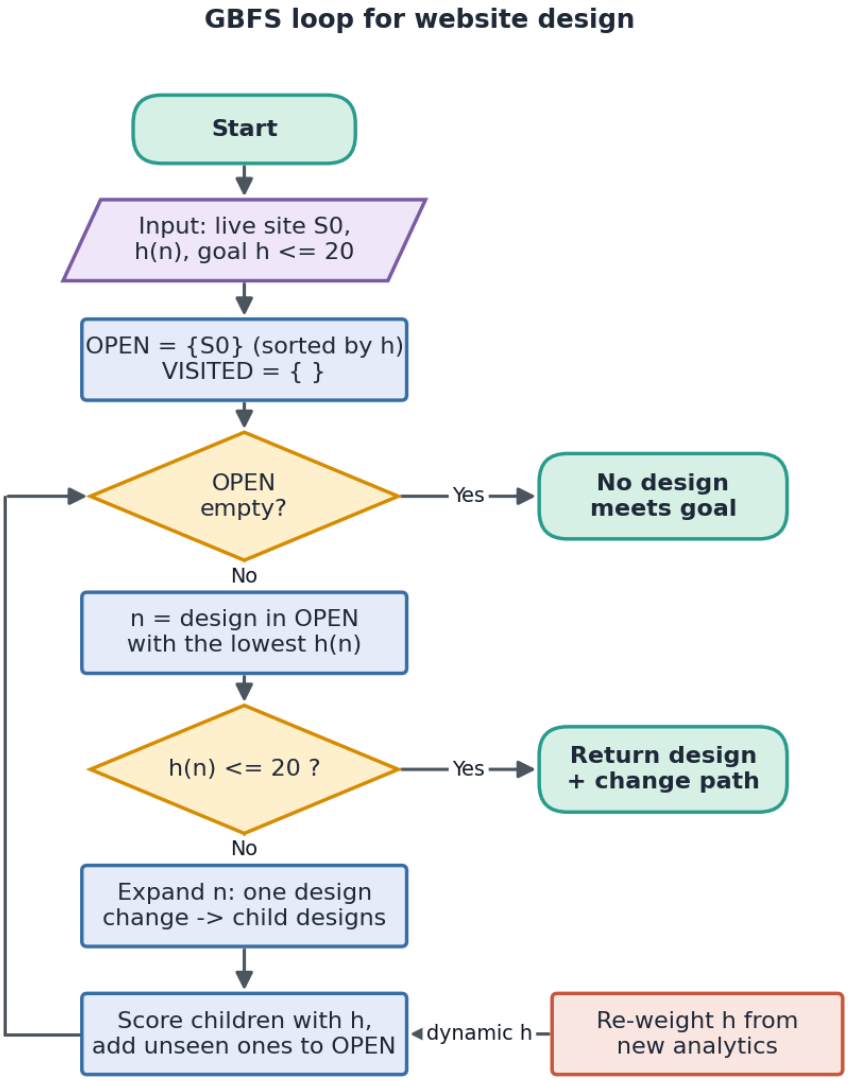


Figure 1. Flowchart of GBFS for website design.

5. Pseudocode

```
function GBFS_WebsiteDesign(S0, h, goal_h):
    OPEN ← priority queue ordered by h(n)          # lowest h comes out first
    OPEN.push(S0);  VISITED ← { }
    while OPEN is not empty:
        n ← OPEN.pop_lowest()
        if n in VISITED: continue
        VISITED.add(n)
        if h(n) ≤ goal_h: return n and its list of changes   # goal test
        for each single design change c in (layout, nav, hero, CTA, images):
            child ← apply(c, n)                          # node expansion
            if child not in VISITED: OPEN.push(child, h(child))
        # optional: re-weight h from fresh analytics (dynamic heuristic)
    return "no design reaches the goal"
```

6. Worked example

Start: classic, mega-menu, slider, no CTA, heavy images, so **h(S0) = 55.8**. Take the first expansion as an example. Switching heavy images (6,1,2) to lazy-loaded (7,10,9) adds 0.5×1 + 0.3×9 + 0.2×7 = 4.6 raw points. That is +9.2 UX points, so h falls to **46.6**. It is the biggest single drop, so GBFS picks it first.

Step	Design change chosen by GBFS	h(n)	Why it was picked
0	start design	55.8	old site
1	images → lazy-loaded	46.6	biggest drop: speed jumps from 1 to 10
2	CTA → sticky	38.6	best child in the new frontier
3	hero → static-image	31.4	slider was slow and weak
4	nav → simple	25.0	usability carries 50% weight
5	**layout → single-page**	**19.4**	**h ≤ 20, so the goal is reached**

GBFS expanded only 5 nodes to reach the goal, out of 3^5 = 243 possible designs.

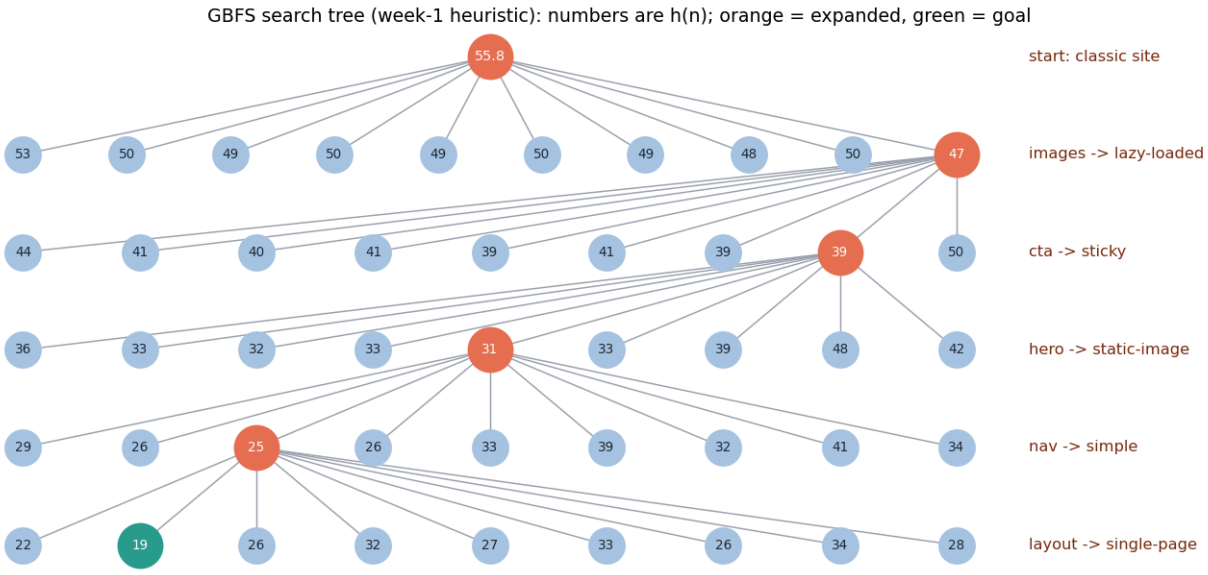


Figure 2. The real search tree from the code. Each row is one expansion (one design change). Blue nodes are alternatives left in OPEN.

7. Convergent vs divergent approaches

	Convergent (plain GBFS)	Divergent (creative extension)
What it does	Always follows the single lowest-h version and narrows to one design	Keeps several promising versions alive and tries unusual changes
How	Pop the best node and stop at the goal	Keep the top-k (beam), add random or novel changes, start from several sites
Strength	Fast, cheap, clear roadmap	Finds designs that GBFS never looks at; gives the designer *choices, not answers*
Risk	Fixation: a misleading h leads to a so-so design	Slower, so it needs a creativity check (novelty + usefulness)

In Figure 2 the blue nodes are the divergent pool and the orange path is the convergent choice. A good design process first diverges (many alternatives), then converges (pick, test, ship).

8. Can the heuristic be dynamic? Yes

User behaviour changes, so the heuristic should change too. Suppose analytics show mobile traffic doubled by week 4. We move weight to mobile: w = (0.35, 0.25, 0.40). Running the same GBFS again gives a **different order and a different final design**: navigation becomes **hamburger** instead of simple, and it is chosen 2nd instead of 4th. In general $h_t(n) = \Sigma\, w_i(t) \cdot f_i(n)$, where the weights $w_i(t)$ are updated from A/B tests, user feedback or a learning agent.

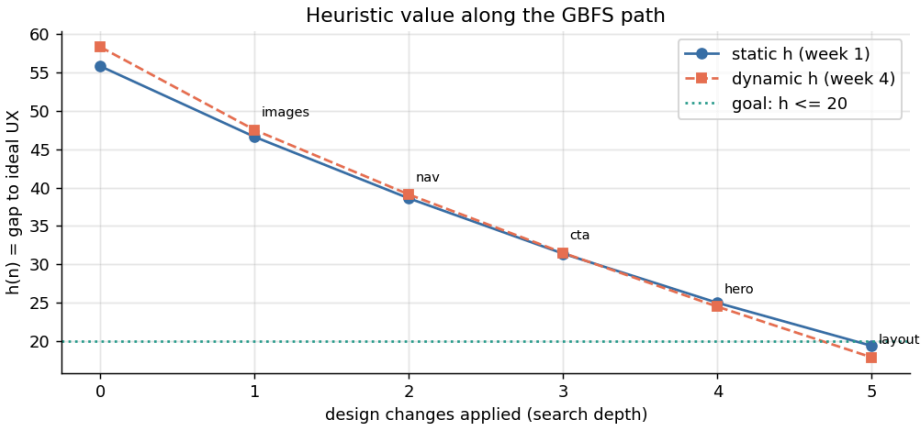


Figure 3. h(n) along the path for both heuristics. The labels show the week-4 order of changes.

9. Extending to chatbot design with multiple trees

A chatbot is a set of conversation trees, for example a **greeting tree, an orders tree and a refunds tree**. Each node is a conversation state, and each child is the next question or reply. GBFS works the same way, with these changes:

- **State:** where the conversation is right now. **Expansion:** the possible next bot turns.
- **h(n):** the expected number of turns left before the user's problem is solved, or 1 − P(resolved).
- **Multiple trees:** keep **one shared OPEN list** for all trees. GBFS then picks the most promising branch from whichever tree fits best, so it can even jump from the orders tree to the refunds tree.

Can every member of a population be a decision tree (Diagram 1)? Yes

Diagram 1 is a tree: a root question, then branches, then more branches, then leaves. That is exactly a chatbot decision tree. **Internal nodes** are tests on the user's message ("mentions refund?", "sounds angry?"), and **leaves** are replies (refund flow, track order, hand over to a human). A genetic algorithm can evolve a whole population of these trees:

- **Crossover** = swap a sub-tree between two parent trees.
- **Mutation** = replace a branch, or change a question or reply.
- **Fitness** = accuracy on sample chats minus a small penalty for big trees.

```
population ← 40 random decision trees (depth ≤ 3)
repeat for G generations:
    fitness(tree) = accuracy on sample chats − 0.003 × tree size
    keep the 2 best trees (elitism)
    while the new population is not full:
        p1, p2 ← tournament_selection(population)
        child ← subtree_swap(p1, p2)                    # crossover
        with probability 0.3: child ← replace_random_branch(child)   # mutation
return best tree
```

In our run, accuracy went from **51% to 93%** in 60 generations. The best tree (Figure 4) learned rules such as "refund and angry → human agent" and "order → track order". One leftover quirk is visible: it asks "refund?" twice. Evolution keeps whatever works, so a cleanup step can simplify the tree afterwards.

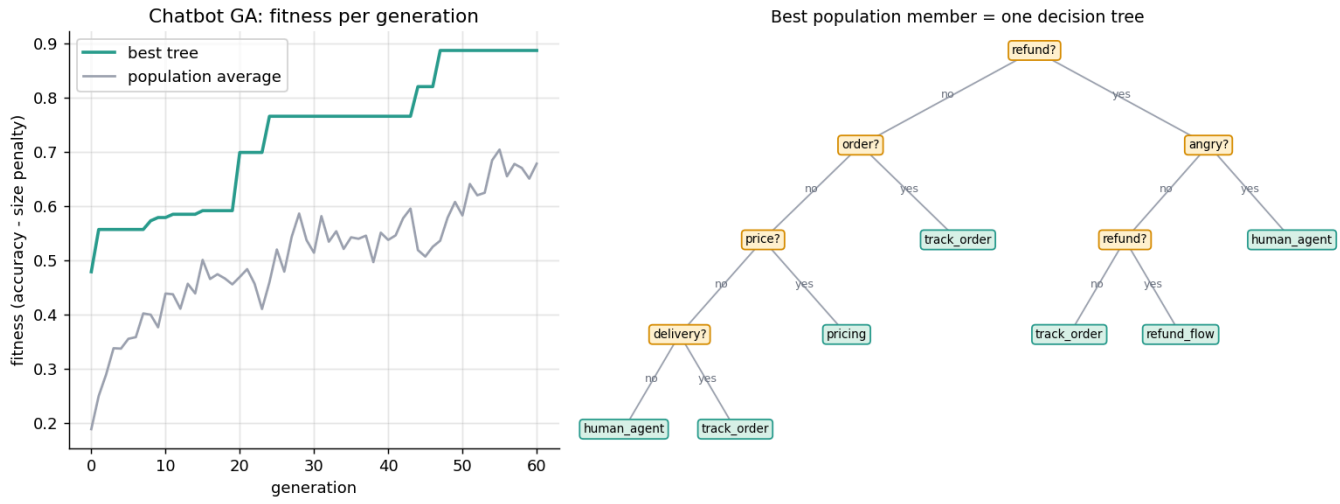


Figure 4. Left: fitness over generations. Right: the best population member, a single decision tree in the shape of Diagram 1.

10. Code and real run

The full program (`q01_code.py`) runs GBFS with both heuristics and the chatbot GA, and draws every chart in this answer.

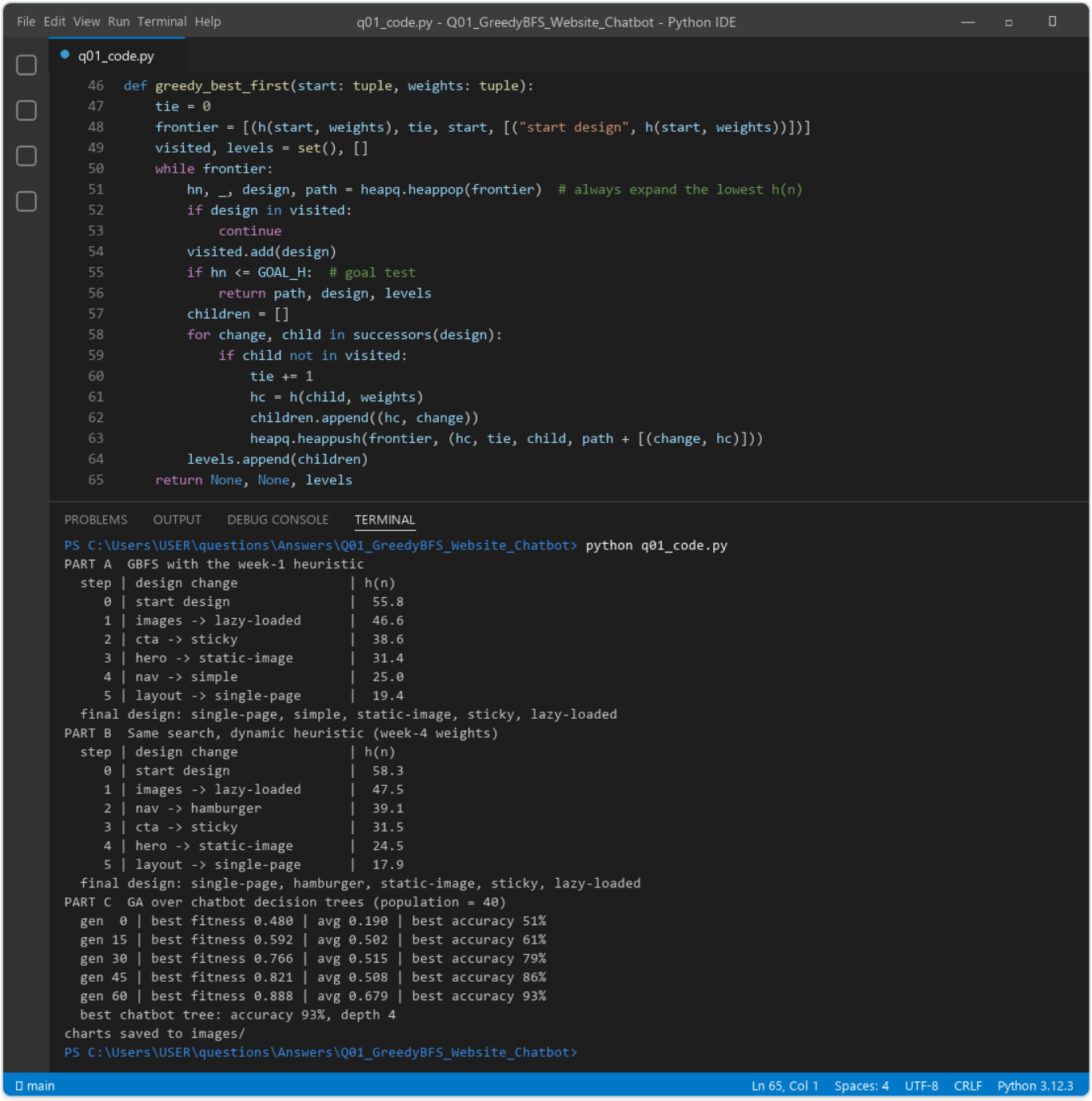


Figure 5. The core GBFS function (lines 46-65) and the real terminal output of the full run.

11. Summary

> GBFS turns website redesign into a guided search: one node = one site version, one expansion = one design change, and h(n) = distance from an ideal UX score. It is fast but greedy, so it can settle on a "good enough" design. Making h dynamic keeps it in step with real users. Moving to a forest of chatbot trees, evolved as a population, adds the divergent, creative side.

Files in this folder: `q01_code.py` (source), `q01_code.ipynb` (Colab/Jupyter version, already run), `q01_output.txt` (console output), `images/` (all figures).